Implementing a GPT model from scratch to generate text

Aristeidis Pagonopoulos



Department of Informatics, Post Graduate Studies program
Artificial Intelligence
Aristotle University of Thessaloniki

June 23, 2026

Overview

- We implement a GPT-124M model from scratch in PyTorch

Overview

- We implement a GPT-124M model from scratch in PyTorch
- Three core components are examined:

Overview

- We implement a GPT-124M model from scratch in PyTorch
- Three core components are examined:
  - **Layer Normalization** — stabilizing activations across the network

Overview

- We implement a GPT-124M model from scratch in PyTorch
- Three core components are examined:
  - **Layer Normalization** — stabilizing activations across the network
  - **Skip (Residual) Connections** — enabling gradient flow in deep networks

Overview

- We implement a GPT-124M model from scratch in PyTorch
- Three core components are examined:
  - **Layer Normalization** — stabilizing activations across the network
  - **Skip (Residual) Connections** — enabling gradient flow in deep networks
  - **Transformer Block** — the forward pass and parameter budget

Overview

- We implement a GPT-124M model from scratch in PyTorch
- Three core components are examined:
  - **Layer Normalization** — stabilizing activations across the network
  - **Skip (Residual) Connections** — enabling gradient flow in deep networks
  - **Transformer Block** — the forward pass and parameter budget
- Configuration used throughout:

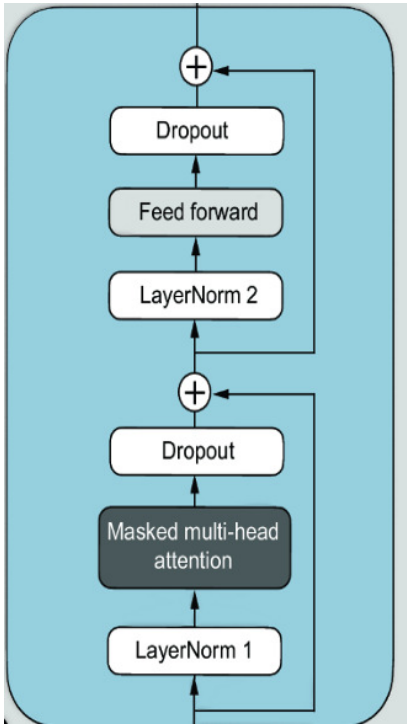
Overview

- We implement a GPT-124M model from scratch in PyTorch
- Three core components are examined:
  - **Layer Normalization** — stabilizing activations across the network
  - **Skip (Residual) Connections** — enabling gradient flow in deep networks
  - **Transformer Block** — the forward pass and parameter budget
- Configuration used throughout:
  - Embedding dimension $d = 768$, $n_{\text{heads}} = 12$, $n_{\text{layers}} = 12$

Overview

- We implement a GPT-124M model from scratch in PyTorch
- Three core components are examined:
  - **Layer Normalization** — stabilizing activations across the network
  - **Skip (Residual) Connections** — enabling gradient flow in deep networks
  - **Transformer Block** — the forward pass and parameter budget
- Configuration used throughout:
  - Embedding dimension $d = 768$, $n_{\text{heads}} = 12$, $n_{\text{layers}} = 12$
  - Vocabulary size = 50,257, context length = 1,024

Transformer Block Overview



Layer Normalization — What & Why

- Given an input vector $\mathbf{x} \in \mathbb{R}^d$, **LayerNorm** computes:

$$\hat{\mathbf{x}} = \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \varepsilon}}, \qquad \mathbf{y} = \gamma \odot \hat{\mathbf{x}} + \beta \tag{1}$$

where $\mu = \frac{1}{d}\sum_{j=1}^{d} x_j, \quad \sigma^2 = \frac{1}{d}\sum_{j=1}^{d}(x_j - \mu)^2$

Layer Normalization — What & Why

- Given an input vector $\mathbf{x} \in \mathbb{R}^d$, **LayerNorm** computes:

$$\hat{\mathbf{x}} = \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \varepsilon}}, \qquad \mathbf{y} = \gamma \odot \hat{\mathbf{x}} + \beta \tag{1}$$

  where $\mu = \frac{1}{d}\sum_{j=1}^{d} x_j, \quad \sigma^2 = \frac{1}{d}\sum_{j=1}^{d}(x_j - \mu)^2$

- γ (scale) and β (shift) are *learnable* parameters; $\varepsilon = 10^{-5}$ prevents division by zero

Layer Normalization — What & Why

- Given an input vector $\mathbf{x} \in \mathbb{R}^d$, **LayerNorm** computes:

$$\hat{\mathbf{x}} = \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \varepsilon}}, \qquad \mathbf{y} = \gamma \odot \hat{\mathbf{x}} + \beta \tag{1}$$

  where $\mu = \frac{1}{d}\sum_{j=1}^{d} x_j, \quad \sigma^2 = \frac{1}{d}\sum_{j=1}^{d}(x_j - \mu)^2$

- γ (scale) and β (shift) are *learnable* parameters; $\varepsilon = 10^{-5}$ prevents division by zero
- Normalization is performed **across features** for each sample independently

Layer Normalization — What & Why

- Given an input vector $\mathbf{x} \in \mathbb{R}^d$, **LayerNorm** computes:

$$\hat{\mathbf{x}} = \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \varepsilon}}, \qquad \mathbf{y} = \gamma \odot \hat{\mathbf{x}} + \beta \tag{1}$$

  where $\mu = \frac{1}{d}\sum_{j=1}^{d} x_j, \quad \sigma^2 = \frac{1}{d}\sum_{j=1}^{d}(x_j - \mu)^2$

- γ (scale) and β (shift) are *learnable* parameters; $\varepsilon = 10^{-5}$ prevents division by zero
- Normalization is performed **across features** for each sample independently
  - Contrast with **BatchNorm**: normalizes across the batch dimension

Layer Normalization — What & Why

- Given an input vector $\mathbf{x} \in \mathbb{R}^d$, **LayerNorm** computes:

$$\hat{\mathbf{x}} = \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \varepsilon}}, \qquad \mathbf{y} = \gamma \odot \hat{\mathbf{x}} + \beta \tag{1}$$

  where $\mu = \frac{1}{d}\sum_{j=1}^{d} x_j, \quad \sigma^2 = \frac{1}{d}\sum_{j=1}^{d}(x_j - \mu)^2$

- γ (scale) and β (shift) are *learnable* parameters; $\varepsilon = 10^{-5}$ prevents division by zero
- Normalization is performed **across features** for each sample independently
  - Contrast with **BatchNorm**: normalizes across the batch dimension
  - LayerNorm does not depend on batch size

LayerNorm vs. RMSNorm

- **LayerNorm** (our implementation, GPT-2 style):

$$\hat{x}_j = \frac{x_j - \mu}{\sqrt{\sigma^2 + \varepsilon}}, \qquad \text{learnable: } \gamma, \beta \in \mathbb{R}^d \tag{2}$$

Used in: GPT-2, BERT, original GPT.
Number of parameters: $2d$, where d is input dimension.

LayerNorm vs. RMSNorm

- **LayerNorm** (our implementation, GPT-2 style):

$$\hat{x}_j = \frac{x_j - \mu}{\sqrt{\sigma^2 + \varepsilon}}, \qquad \text{learnable: } \gamma, \beta \in \mathbb{R}^d \tag{2}$$

  Used in: GPT-2, BERT, original GPT.
  Number of parameters: $2d$, where d is input dimension.
- **RMSNorm** (modern alternative):

$$\hat{x}_j = \frac{x_j}{\text{RMS}(\mathbf{x})}, \quad \text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d}\sum_j x_j^2 + \varepsilon} \tag{3}$$

  Used in: LLaMA, Mistral, Gemma, DeepSeek.
  Number of parametrs: d.

LayerNorm vs. RMSNorm

- **LayerNorm** (our implementation, GPT-2 style):

$$\hat{x}_j = \frac{x_j - \mu}{\sqrt{\sigma^2 + \varepsilon}}, \qquad \text{learnable: } \gamma, \beta \in \mathbb{R}^d \tag{2}$$

  Used in: GPT-2, BERT, original GPT.
  Number of parameters: $2d$, where d is input dimension.
- **RMSNorm** (modern alternative):

$$\hat{x}_j = \frac{x_j}{\text{RMS}(\mathbf{x})}, \quad \text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d}\sum_j x_j^2 + \varepsilon} \tag{3}$$

  Used in: LLaMA, Mistral, Gemma, DeepSeek.
  Number of parametrs: d.
- The difference is that RMSNorm **skips centering** (no mean subtraction)

LayerNorm vs. RMSNorm

- **LayerNorm** (our implementation, GPT-2 style):

$$\hat{x}_j = \frac{x_j - \mu}{\sqrt{\sigma^2 + \varepsilon}}, \qquad \text{learnable: } \gamma, \beta \in \mathbb{R}^d \tag{2}$$

  Used in: GPT-2, BERT, original GPT.
  Number of parameters: $2d$, where d is input dimension.
- **RMSNorm** (modern alternative):

$$\hat{x}_j = \frac{x_j}{\text{RMS}(\mathbf{x})}, \quad \text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d}\sum_j x_j^2 + \varepsilon} \tag{3}$$

  Used in: LLaMA, Mistral, Gemma, DeepSeek.
  Number of parametrs: d.
- The difference is that RMSNorm **skips centering** (no mean subtraction)

LayerNorm vs. RMSNorm

- **LayerNorm** (our implementation, GPT-2 style):

$$\hat{x}_j = \frac{x_j - \mu}{\sqrt{\sigma^2 + \varepsilon}}, \qquad \text{learnable: } \gamma, \beta \in \mathbb{R}^d \tag{2}$$

  Used in: GPT-2, BERT, original GPT.
  Number of parameters: $2d$, where d is input dimension.
- **RMSNorm** (modern alternative):

$$\hat{x}_j = \frac{x_j}{\text{RMS}(\mathbf{x})}, \quad \text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d}\sum_j x_j^2 + \varepsilon} \tag{3}$$

  Used in: LLaMA, Mistral, Gemma, DeepSeek.
  Number of parametrs: d.
- The difference is that RMSNorm **skips centering** (no mean subtraction)

Pre-Norm vs. Post-Norm

- **Post-Norm** (original Transformer, Vaswani et al. 2017):

$$\mathbf{x} \leftarrow \text{LayerNorm}(\mathbf{x} + \text{SubLayer}(\mathbf{x})) \tag{4}$$

Pre-Norm vs. Post-Norm

- **Post-Norm** (original Transformer, Vaswani et al. 2017):

$$\mathbf{x} \leftarrow \text{LayerNorm}(\mathbf{x} + \text{SubLayer}(\mathbf{x})) \tag{4}$$

  - Norm is applied *after* the residual addition

Pre-Norm vs. Post-Norm

- **Post-Norm** (original Transformer, Vaswani et al. 2017):

$$\mathbf{x} \leftarrow \text{LayerNorm}(\mathbf{x} + \text{SubLayer}(\mathbf{x})) \tag{4}$$

  - Norm is applied *after* the residual addition
  - Unstable for very deep networks

Pre-Norm vs. Post-Norm

- **Post-Norm** (original Transformer, Vaswani et al. 2017):

$$\mathbf{x} \leftarrow \text{LayerNorm}(\mathbf{x} + \text{SubLayer}(\mathbf{x})) \tag{4}$$

  - Norm is applied *after* the residual addition
  - Unstable for very deep networks
- **Pre-Norm** (our implementation, GPT-2 style):

$$\mathbf{x} \leftarrow \mathbf{x} + \text{SubLayer}(\text{LayerNorm}(\mathbf{x})) \tag{5}$$

Pre-Norm vs. Post-Norm

- **Post-Norm** (original Transformer, Vaswani et al. 2017):

$$\mathbf{x} \leftarrow \text{LayerNorm}(\mathbf{x} + \text{SubLayer}(\mathbf{x})) \tag{4}$$

  - Norm is applied *after* the residual addition
  - Unstable for very deep networks
- **Pre-Norm** (our implementation, GPT-2 style):

$$\mathbf{x} \leftarrow \mathbf{x} + \text{SubLayer}(\text{LayerNorm}(\mathbf{x})) \tag{5}$$

  - Norm is applied *before* the sublayer; residual path stays clean

Pre-Norm vs. Post-Norm

- **Post-Norm** (original Transformer, Vaswani et al. 2017):

$$\mathbf{x} \leftarrow \text{LayerNorm}(\mathbf{x} + \text{SubLayer}(\mathbf{x})) \tag{4}$$

  - Norm is applied *after* the residual addition
  - Unstable for very deep networks
- **Pre-Norm** (our implementation, GPT-2 style):

$$\mathbf{x} \leftarrow \mathbf{x} + \text{SubLayer}(\text{LayerNorm}(\mathbf{x})) \tag{5}$$

  - Norm is applied *before* the sublayer; residual path stays clean
  - More stable training from the start; dominant in modern LLMs

Pre-Norm vs. Post-Norm

- **Post-Norm** (original Transformer, Vaswani et al. 2017):

$$\mathbf{x} \leftarrow \text{LayerNorm}(\mathbf{x} + \text{SubLayer}(\mathbf{x})) \tag{4}$$

  - Norm is applied *after* the residual addition
  - Unstable for very deep networks
- **Pre-Norm** (our implementation, GPT-2 style):

$$\mathbf{x} \leftarrow \mathbf{x} + \text{SubLayer}(\text{LayerNorm}(\mathbf{x})) \tag{5}$$

  - Norm is applied *before* the sublayer; residual path stays clean
  - More stable training from the start; dominant in modern LLMs
  - GPT-2, LLaMA, Mistral, and our code all use Pre-Norm

Skip (Residual) Connections

- Each `TransformerBlock` contains **two** residual additions:

$$\mathbf{x} \leftarrow \mathbf{x} + \text{Dropout}(\text{Attention}(\text{Norm}(\mathbf{x}))) \tag{6}$$

$$\mathbf{x} \leftarrow \mathbf{x} + \text{Dropout}(\text{FFN}(\text{Norm}(\mathbf{x}))) \tag{7}$$

Skip (Residual) Connections

- Each `TransformerBlock` contains **two** residual additions:

$$\mathbf{x} \leftarrow \mathbf{x} + \text{Dropout}(\text{Attention}(\text{Norm}(\mathbf{x}))) \tag{6}$$

$$\mathbf{x} \leftarrow \mathbf{x} + \text{Dropout}(\text{FFN}(\text{Norm}(\mathbf{x}))) \tag{7}$$

- The **shortcut** (identity path) bypasses each sublayer entirely

Skip (Residual) Connections

- Each `TransformerBlock` contains **two** residual additions:

$$\mathbf{x} \leftarrow \mathbf{x} + \text{Dropout}(\text{Attention}(\text{Norm}(\mathbf{x}))) \tag{6}$$

$$\mathbf{x} \leftarrow \mathbf{x} + \text{Dropout}(\text{FFN}(\text{Norm}(\mathbf{x}))) \tag{7}$$

- The **shortcut** (identity path) bypasses each sublayer entirely
- Originally introduced in ResNets (He et al. 2016); adopted in Transformers for the same reasons:

Skip (Residual) Connections

- Each `TransformerBlock` contains **two** residual additions:

$$\mathbf{x} \leftarrow \mathbf{x} + \text{Dropout}(\text{Attention}(\text{Norm}(\mathbf{x}))) \tag{6}$$

$$\mathbf{x} \leftarrow \mathbf{x} + \text{Dropout}(\text{FFN}(\text{Norm}(\mathbf{x}))) \tag{7}$$

- The **shortcut** (identity path) bypasses each sublayer entirely
- Originally introduced in ResNets (He et al. 2016); adopted in Transformers for the same reasons:
  - **Gradient highway**: gradients flow directly to early layers, preventing vanishing gradients across 12 deep blocks

Skip (Residual) Connections

- Each `TransformerBlock` contains **two** residual additions:

$$\mathbf{x} \leftarrow \mathbf{x} + \text{Dropout}(\text{Attention}(\text{Norm}(\mathbf{x}))) \tag{6}$$

$$\mathbf{x} \leftarrow \mathbf{x} + \text{Dropout}(\text{FFN}(\text{Norm}(\mathbf{x}))) \tag{7}$$

- The **shortcut** (identity path) bypasses each sublayer entirely
- Originally introduced in ResNets (He et al. 2016); adopted in Transformers for the same reasons:
  - **Gradient highway**: gradients flow directly to early layers, preventing vanishing gradients across 12 deep blocks
  - **Magnitude control**: without skip connections, repeated additions cause exponential growth of activations

Skip Connections — Implementation

- In code, the two skip connections appear explicitly as:

```
def forward(self, x):
    shortcut = x
    x = self.norm1(x)           # Pre-LayerNorm
    x = self.att(x)             # Multi-Head Attention
    x = self.drop_shortcut(x)
    x = x + shortcut            # Skip connection (1)

    shortcut = x
    x = self.norm2(x)           # Pre-LayerNorm
    x = self.ff(x)              # FeedForward
    x = self.drop_shortcut(x)
    x = x + shortcut            # Skip connection (2)
    return x
```

Skip Connections — Implementation

- In code, the two skip connections appear explicitly as:

```
def forward(self, x):
    shortcut = x
    x = self.norm1(x)           # Pre-LayerNorm
    x = self.att(x)             # Multi-Head Attention
    x = self.drop_shortcut(x)
    x = x + shortcut            # Skip connection (1)

    shortcut = x
    x = self.norm2(x)           # Pre-LayerNorm
    x = self.ff(x)              # FeedForward
    x = self.drop_shortcut(x)
    x = x + shortcut            # Skip connection (2)
    return x
```

- Dropout is applied **before** the addition (on the residual branch only)

Skip Connections — Implementation

- In code, the two skip connections appear explicitly as:

```
def forward(self, x):
    shortcut = x
    x = self.norm1(x)           # Pre-LayerNorm
    x = self.att(x)             # Multi-Head Attention
    x = self.drop_shortcut(x)
    x = x + shortcut            # Skip connection (1)

    shortcut = x
    x = self.norm2(x)           # Pre-LayerNorm
    x = self.ff(x)              # FeedForward
    x = self.drop_shortcut(x)
    x = x + shortcut            # Skip connection (2)
    return x
```

- Dropout is applied **before** the addition (on the residual branch only)
- The identity path is never zeroed — the model always has a direct gradient path

Transformer Block — Forward Pass

The full forward pass of one `TransformerBlock` proceeds as follows:

1. **Save shortcut:** $\mathbf{s} \leftarrow \mathbf{x}$

Transformer Block — Forward Pass

The full forward pass of one `TransformerBlock` proceeds as follows:

1. **Save shortcut:** $\mathbf{s} \leftarrow \mathbf{x}$
2. **Attention sub-layer:**
   $\mathbf{x} \leftarrow \text{Norm}_1(\mathbf{x}) \rightarrow \text{MHA}(\mathbf{x}) \rightarrow \text{Dropout}(\mathbf{x})$

Transformer Block — Forward Pass

The full forward pass of one `TransformerBlock` proceeds as follows:

1. **Save shortcut:** $\mathbf{s} \leftarrow \mathbf{x}$
2. **Attention sub-layer:**
   $\mathbf{x} \leftarrow \mathrm{Norm}_1(\mathbf{x}) \rightarrow \mathrm{MHA}(\mathbf{x}) \rightarrow \mathrm{Dropout}(\mathbf{x})$
3. **First residual add:** $\mathbf{x} \leftarrow \mathbf{x} + \mathbf{s}$

Transformer Block — Forward Pass

The full forward pass of one `TransformerBlock` proceeds as follows:

1. **Save shortcut:** $\mathbf{s} \leftarrow \mathbf{x}$
2. **Attention sub-layer:**
   $\mathbf{x} \leftarrow \mathrm{Norm}_1(\mathbf{x}) \rightarrow \mathrm{MHA}(\mathbf{x}) \rightarrow \mathrm{Dropout}(\mathbf{x})$
3. **First residual add:** $\mathbf{x} \leftarrow \mathbf{x} + \mathbf{s}$
4. **Save shortcut:** $\mathbf{s} \leftarrow \mathbf{x}$

Transformer Block — Forward Pass

The full forward pass of one `TransformerBlock` proceeds as follows:

1. **Save shortcut:** $\mathbf{s} \leftarrow \mathbf{x}$
2. **Attention sub-layer:**
   $\mathbf{x} \leftarrow \mathrm{Norm}_1(\mathbf{x}) \rightarrow \mathrm{MHA}(\mathbf{x}) \rightarrow \mathrm{Dropout}(\mathbf{x})$
3. **First residual add:** $\mathbf{x} \leftarrow \mathbf{x} + \mathbf{s}$
4. **Save shortcut:** $\mathbf{s} \leftarrow \mathbf{x}$
5. **FeedForward sub-layer:**
   $\mathbf{x} \leftarrow \mathrm{Norm}_2(\mathbf{x}) \rightarrow \mathrm{FFN}(\mathbf{x}) \rightarrow \mathrm{Dropout}(\mathbf{x})$

Transformer Block — Forward Pass

The full forward pass of one `TransformerBlock` proceeds as follows:

1. **Save shortcut:** $\mathbf{s} \leftarrow \mathbf{x}$
2. **Attention sub-layer:**
   $\mathbf{x} \leftarrow \mathrm{Norm}_1(\mathbf{x}) \rightarrow \mathrm{MHA}(\mathbf{x}) \rightarrow \mathrm{Dropout}(\mathbf{x})$
3. **First residual add:** $\mathbf{x} \leftarrow \mathbf{x} + \mathbf{s}$
4. **Save shortcut:** $\mathbf{s} \leftarrow \mathbf{x}$
5. **FeedForward sub-layer:**
   $\mathbf{x} \leftarrow \mathrm{Norm}_2(\mathbf{x}) \rightarrow \mathrm{FFN}(\mathbf{x}) \rightarrow \mathrm{Dropout}(\mathbf{x})$
6. **Second residual add:** $\mathbf{x} \leftarrow \mathbf{x} + \mathbf{s}$

Transformer Block — Forward Pass

The full forward pass of one `TransformerBlock` proceeds as follows:

1. **Save shortcut:** $\mathbf{s} \leftarrow \mathbf{x}$
2. **Attention sub-layer:**
   $\mathbf{x} \leftarrow \text{Norm}_1(\mathbf{x}) \rightarrow \text{MHA}(\mathbf{x}) \rightarrow \text{Dropout}(\mathbf{x})$
3. **First residual add:** $\mathbf{x} \leftarrow \mathbf{x} + \mathbf{s}$
4. **Save shortcut:** $\mathbf{s} \leftarrow \mathbf{x}$
5. **FeedForward sub-layer:**
   $\mathbf{x} \leftarrow \text{Norm}_2(\mathbf{x}) \rightarrow \text{FFN}(\mathbf{x}) \rightarrow \text{Dropout}(\mathbf{x})$
6. **Second residual add:** $\mathbf{x} \leftarrow \mathbf{x} + \mathbf{s}$

- The FeedForward network expands to $4d$ hidden units:
  $\text{FFN}(\mathbf{x}) = \text{GELU}(\mathbf{x}\,\mathbf{W}_1 + \mathbf{b}_1)\,\mathbf{W}_2 + \mathbf{b}_2$

Parameter Count per Transformer Block

- With embedding dimension $d = 768$:

Component	**Formula**	**# Parameters**
Multi-Head Attention	$4d^2 = 4 \times 768^2$	2,359,296
FeedForward ($d \to 4d \to d$)	$8d^2 + 5d = 8 \times 768^2 + 5 \times 768$	4,722,432
LayerNorm $\times 2$ ($\gamma + \beta$)	$4d = 4 \times 768$	3,072
Total per block		$\approx$ **7.08 M**
Total (12 blocks)	12×7.08 M	$\approx$ **84.9 M**

Table: Parameter breakdown for GPT-124M ($d = 768$, $n_{\text{layers}} = 12$).

Parameter Count per Transformer Block

- With embedding dimension $d = 768$:

Component	**Formula**	**# Parameters**
Multi-Head Attention	$4d^2 = 4 \times 768^2$	2,359,296
FeedForward ($d \to 4d \to d$)	$8d^2 + 5d = 8 \times 768^2 + 5 \times 768$	4,722,432
LayerNorm $\times 2$ ($\gamma + \beta$)	$4d = 4 \times 768$	3,072
Total per block		$\approx$ **7.08 M**
Total (12 blocks)	12×7.08 M	$\approx$ **84.9 M**

Table: Parameter breakdown for GPT-124M ($d = 768$, $n_{\text{layers}} = 12$).

- Remaining parameters: token embeddings ($50{,}257 \times 768 \approx 38.6$ M) and position embeddings ($1{,}024 \times 768 \approx 0.8$ M)

Conclusion

- **Layer Normalization**: stabilizes training by normalising per-sample feature distributions; Pre-Norm placement is more stable than Post-Norm for deep networks

Conclusion

- **Layer Normalization**: stabilizes training by normalising per-sample feature distributions; Pre-Norm placement is more stable than Post-Norm for deep networks
- **RMSNorm** is the efficient modern alternative — same effect with half the parameters and no centering step

Conclusion

- **Layer Normalization**: stabilizes training by normalising per-sample feature distributions; Pre-Norm placement is more stable than Post-Norm for deep networks
- **RMSNorm** is the efficient modern alternative — same effect with half the parameters and no centering step
- **Skip connections** provide a gradient highway across all layers and encourage each sublayer to learn only incremental corrections

Conclusion

- **Layer Normalization**: stabilizes training by normalising per-sample feature distributions; Pre-Norm placement is more stable than Post-Norm for deep networks
- **RMSNorm** is the efficient modern alternative — same effect with half the parameters and no centering step
- **Skip connections** provide a gradient highway across all layers and encourage each sublayer to learn only incremental corrections
- **Transformer Block forward pass**: two symmetric sublayers (Attention, FFN), each wrapped in Pre-Norm and a residual addition, for a total of $\approx 7.08\,\text{M}$ parameters per block

Thank you